K Flex Codebase Review, Gap Analysis and Implementation Plan

Oct 23, 2026. One document: what the code does today (sections 2 to 4), decisions and their defaults (5), rules for every change (6), the target data model (7), the phased plan (8), test scenarios (9), answers for the owner (10), and a verification log (Appendix A). A model with no context should read section 2, then section 6, then do the phases in section 8 in order.

1. Summary and verdict

The marketplace core works, but most of the owner's October requirements do not exist yet, and three security gaps must be closed before anything is built on top of it. This document records what the code does today, what is missing, and a step-by-step plan that keeps the live app working throughout. Payments are the last phase.

Basis of this review. The public repo garimasharmabridgeflex-ops/Bridge-Flex at commit `3a79a61`: both backend services, the Firestore and Storage rules, deploy scripts, CI workflows and the Flutter client. The main findings were re-checked against a fresh clone on Oct 23, 2026 (Appendix A). The app was not run and production was not touched; anything about the live project is marked unverified.

Working rule: do not wait for the owner. Every owner decision in section 5 has a safe default, and every new gate or feature ships behind a flag (section 6.1). So the order is build on the default, keep the app working at every step, and change it when the owner answers. Most owner answers become admin-editable settings (prices, rates, plans, policy text and fees, company details), not code changes. The few that change structure are marked in section 5. Two things are never published on a default: legal text and company or VAT numbers, because the owner asked for that explicitly.

The eight findings that matter most

1. **Four real bugs** in shift filtering, training badges, the course counter and My Shifts (section 3.1). Cheap to fix.
2. **Firestore rules let a user set their own trust fields.** A client can write `identityVerified`, `ofstedVerified`, `rightToWorkVerified`, `suspended` and even `role` on its own profile, and can create a profile with `dbsStatus: verified`.

Nurseries rely on these badges. ARCHITECTURE.md says they are frozen; the rules file does not freeze them.

3. **Clients can write shifts directly**, bypassing `CreateShift`, because the rules allow it. Every server gate we add (nursery approval, trial expiry, vetting) can be bypassed until the rules are closed. The app itself never writes shifts directly, so closing them is safe.
4. **Approving any DBS document marks the practitioner `dbsStatus: verified`** (`documents.go`). The owner's rule is that a DBS number or upload must never verify anyone; only the full admin checklist does.
5. **Nothing stops an unvetted practitioner from applying, or a nursery from approving them.** The only server gate is an optional training check that is switched off in production.
6. **Process risk is bigger than architecture risk.** The backend exists as two hand-synced copies of every file, there is no backend CI, no staging project, and deploys run from a laptop with `gcloud`.
7. **Money is not modelled.** There is one `payRate` (a float, typed by the nursery). There is no split between what the nursery is charged and what the practitioner earns, no pence-based amounts, and no stored "completed" state: it is derived at read time from the end time.
8. **Missing plumbing:** no scheduler (needed for trial and recheck reminders), no email sending, no audit log, and one all-powerful admin claim.

Microservices: keep what you have

Keep the three existing Cloud Functions services: `core`, `communication` and `payments`. Splitting further adds network failure modes, distributed transactions and cost, with no benefit at this scale. The existing split already isolates the Stripe SDK and gives independent deploys. Treat `core` as a modular monolith organised by domain file. The real structural fixes are the duplicated source trees, CI and a staging project, all in Phase 0.

Recommended order

Phase 0 safety net, then fixes and rules, then platform foundations, legal acceptance, vetting, nursery approval, shifts and rates, timesheets, payroll, subscriptions, and finally Stripe. The owner can launch before Stripe: plans can be assigned by an admin and invoices settled by bank transfer. See section 8.

2. How the system works today

K Flex (the repo and code call it Bridge Flex) is a Flutter app on top of Firebase and three Go Cloud Functions services, all in one production project, `kvision-503115`, region `europe-west2`. There is no staging project.

Stack

- **Client:** Flutter app in `frontend/` (Riverpod for state, go_router for navigation). Version `1.6.0+9`. CI builds with Flutter 3.44.x.
- **Backend:** Go services in `backend/services/` on Cloud Functions 2nd gen (runs on Cloud Run), using `functions-framework-go`. `go.mod` says Go 1.25.8; deploy scripts use runtime `go125`.
- **Shared Go module:** `backend/shared` holds the auth middleware (`auth`), the JSON and error helpers (`httpjson`) and the Firebase app factory (`fbapp`). Each service bundles its own copy.
- **Firebase products:** Auth (email/password, Google, Apple), Firestore, Storage, Pub/Sub, FCM push, Remote Config, and Hosting (`backend/public` serves the APK download page and the legal pages).

Services

Service	Purpose	Notes
`core`	Profiles, shifts, booking, ratings, documents, admin, training	About 35 functions. Four are event triggers: `SyncProfilePublic`, `RecomputeRating`, `MatchNewShift`, `InitProfileOnSignUp`. The last cannot deploy (no Auth Eventarc provider in the project); `deploy.sh` ignores the failure and the client creates the profile instead.
`communication`	Chat, in-app notifications, FCM push	Six Pub/Sub triggers on topics `shift-booked`, `shift-applied`, `shift-application-decided`, `rating-received`, `shift-matched`, `shift-cancelled`.

Service	Purpose	Notes
`payments`	Reserved for Stripe	One stub, `CreatePayout`, always returns 501 `PAYMENTS_NOT_YET_BUILT`. No Stripe SDK or keys exist.

How a request flows

1. The app calls an HTTPS function with `Authorization: Bearer <Firebase ID token>`. The client wrapper is `frontend/lib/core/api/api_client.dart` (10 s token timeout, 15 s request timeout). Every endpoint is listed in the `ApiFunction` enum in `frontend/lib/core/config/env.dart`.
2. Each handler is wrapped by `authed()` in `context.go`, which calls `auth.RequireAuth` (`backend/shared/auth/middleware.go`). That verifies the token and puts the uid and claims in the request context.
3. Admin means the Firebase custom claim `admin: true`, read by `isAdmin(ctx)` in `documents.go`. There is only this one admin level.
4. Errors use one envelope: `{"error":{"code":"SHIFT_FULL","message":"..."}}` with a matching HTTP status.
5. All functions are deployed `--allow-unauthenticated` at the IAM layer. That is intentional; the handler rejects unauthenticated callers itself.

Data in Firestore and Storage today

Collection	Written by	What it holds	Client access (per `firestore.rules`)
`profiles/{uid}`	core, plus client at sign-up	Private profile, role `nursery` or `staff`, DBS status, rating, wizard fields. Subcollections `fcmTokens` and `trainingProgress`.	Owner read. Owner update, but only `dbsStatus`, `rating` and training ids are frozen (finding S2).
`profilesPublic/{uid}`	`SyncProfilePublic` trigger	Safe mirror shown to other users, with a coarse geohash instead of exact location.	Any signed-in read. No client write.

Collection	Written by	What it holds	Client access (per `firestore.rules`)
`shifts/{id}`	core	`status` (`open`, `booked`, `cancelled`), `capacity`, `bookedStaffIds` (approved), `pendingStaffIds` (applied), `rejectedStaffIds`, `payRate` (float), `paymentStatus` (unused).	Signed-in read if open. Client create and update allowed (finding S3).
`documents/{id}`	core	Upload metadata and review status. Files live in Storage.	Owner read.
`ratings`, `chatSessions` and its messages, `notifications`	core, communication	Ratings, chat, in-app notifications.	Participant or owner.
`trainingModules`, `platformConfig/training`	core (admin)	9 seeded modules with answer keys. `requiredModuleIds` is empty in production, so there is no training gate.	None (server only).
`transactions`	nobody yet	Name reserved for payments.	None.

Storage paths: `dbs-documents/{uid}/`, `personal-documents/{uid}/{type}/`, `profile-photos/`, `nursery-photos/`, `training-videos/` (admin write). Owner and any admin can read the document paths; any signed-in user can read photos.

The booking flow today

1. A practitioner calls `AcceptShift`. Despite the name, this only applies: it adds the uid to `pendingStaffIds` inside a transaction after a role check and the optional training gate.
2. The nursery calls `ApproveShiftApplicant` or `RejectShiftApplicant`. Approval runs in a transaction, checks capacity, and auto-rejects the remaining applicants once the shift is full. It then publishes `shift-booked`, which creates the chat session.

3. `CancelShift`: a practitioner dropping out reopens the shift; a nursery cancelling sets it to `cancelled`. There is no policy, reason or timestamp. `MarkNoShow` records no-shows.
4. "Completed" is not stored. `stats.go` derives it as: not cancelled, past the end time, at least one booked practitioner.
5. Matching (`MatchNewShift`) publishes one message per practitioner profile for every new shift: no radius, no vetting or suspension filter.

Client structure

Features live in `frontend/lib/features/`: admin, auth, chat, documents, home, notifications, onboarding (a staff wizard and a nursery wizard), profile, ratings, settings, shifts and training. The router gate in `app/router.dart` sends admin accounts only to `/admin`, and everyone else through role selection and `hasCompletedBasics`. The admin panel is inside the mobile app, with four tabs (Overview, Queue, Users, Training). The client writes to Firestore directly in exactly one place, `auth_repository.dart`: it creates the profile at sign-up and merges `photoUrl`. Everything else goes through the HTTP functions.

Release and operations today

- **Backend deploy:** per-function `gcloud functions deploy` via `backend/services/<service>/deploy.sh`, run by hand with `PROJECT_ID` set. One-time project setup is in `backend/PRODUCTION_SETUP.md`.
- **Two copies of every backend file.** Each service has its source at the service root (needed by the Cloud Functions buildpack) and again under `function/` (used by local dev). Any change must be made in both. Today they are byte-identical. Nothing enforces it.
- **Rules and indexes:** `backend/firestore.rules`, `storage.rules`, `firestore.indexes.json`, wired through `firebase.json`.
- **Tests:** Go trigger tests per service; 22 Firestore rules tests (`backend/rules-tests`, Vitest). There is no `frontend/test` folder.
- **CI:** GitHub Actions build Android (APK and AAB, via Workload Identity Federation), iOS (TestFlight upload, gated on four missing secrets) and screenshots. No backend CI and no deploy CI.

- **App update gate:** `frontend/lib/core/update/app_update_service.dart` reads Remote Config keys `latest_version_code`, `min_supported_version_code` and `download_url`. Raising `min_supported_version_code` forces old builds to update. This is the main tool for retiring old app versions.
- **Gaps:** no staging, no monitoring or alerts, backups unverified, no Cloud Scheduler, no email service, and a leftover test admin account that must be rotated.

3. Verified defects and risks

Four bugs, nine security findings and eleven reliability gaps. File paths are relative to `backend/services/core/` for Go and `frontend/lib/features/` for Dart. Line numbers were taken at commit `3a79a61` and will drift. Items marked **confirmed** were re-read in the code on Oct 23, 2026 (Appendix A); the rest come from the original review.

3.1 Bugs

ID	What users see	Cause	Fix
B1	Browse Shifts shows odd blank gaps, never shows "No matches" with Near me on, shows every shift when location is unknown, and shows stale Apply buttons after applying.	The Near me filter runs inside each card (`_NearMeAwareCard`, `shifts/presentation/browse_shifts_screen.dart` about line 878), not in `_applyFilters` (about line 131). Non-matching cards render `SizedBox.shrink()` but the list still adds separators, and `filtered.isEmpty` is never true. `shift_detail_screen.dart` only invalidates `shiftDetailProvider`. (Both functions confirmed present; behaviour not run.)	Add an additive `nurseryArea` field to each shift in `ListOpenShifts`, filter inside `_applyFilters`, show a snackbar when location is missing, and invalidate `openShiftsProvider` and `myShiftsProvider` after applying.
B2	A nursery may see 0 completed training modules on an applicant.	`buildPublicFromProfile` (`profiles.go` about line 600) omits `TrainingCompletedModuleIDs`. It is called by `updateProfile` (about line 427, a full Set whose error is ignored) and by the `GetPublicProfile` fallback (about line 583). The `SyncProfilePublic` trigger does include the field (about line 139) and probably restores it moments later, so this is a race, not a permanent wipe. **Confirmed.**	Add the missing assignment. Better: remove the duplicate builder, or add a parity test that fails when the two builders differ.

ID	What users see	Cause	Fix
B3	Course counter can read "5 of 2".	`completedCount(progress)` in `training.go` (about line 648) counts every progress record, including modules no longer published. **Confirmed.**	Count only module ids present in the published list.
B4	A practitioner's applications vanish from My Shifts.	`listMyShifts` (`shifts.go` about lines 317 to 350) queries only `bookedStaffId` and `bookedStaffIds`, not `pendingStaffIds`. Query errors are swallowed with `break`. **Confirmed.**	Add a third query on `pendingStaffIds`, return an error instead of swallowing it, and add an "Applied, awaiting approval" badge on the card.

One minor item: `training/presentation/training_quiz_screen.dart` (line 80) invalidates `trainingOverviewProvider` but not `ownProfileProvider`. It is not confirmed that any screen reads training state from the profile, so treat it as low priority.

3.2 Security

ID	Finding	Evidence	Fix
S1	Approving any DBS document sets `dbsStatus: verified`. Conflicts with the owner's rule. **Confirmed.**	`documents.go` (about lines 318 to 328), `reviewDocument`. The same function sets `rightToWorkVerified` on a right-to-work approval.	Stop deriving status from document approval. Status comes only from the vetting checklist (Phase 4).

ID	Finding	Evidence	Fix
S2	A user can write their own trust fields. **Confirmed.**	`firestore.rules` `profiles/{uid}`: update freezes only `dbsStatus`, `rating` and `trainingCompletedModuleIds`. Not frozen: `role`, `suspended`, `identityVerified`, `ofstedVerified`, `rightToWorkVerified`. Create forbids only `trainingCompletedModuleIds`, so `dbsStatus: verified` is accepted on first write. These fields mirror into `profilesPublic`. `ARCHITECTURE.md` (lines 176 to 182) says they are frozen. `admin.go` already has a server-side write path for the two badges, which the rules let clients skip. The rules tests cover only `dbsStatus`.	Replace with an allowlist. See Phase 1 for the exact rule and why it is safe for current clients.
S3	Shifts can be created and edited straight in Firestore. **Confirmed.**	`firestore.rules` `shifts/{id}`: create and update allowed for the owning nursery. Update freezes only `status` and `bookedStaffId`, so `bookedStaffIds`, `pendingStaffIds`, `capacity` and `payRate` are writable. The app never does this; all shift writes use HTTP (`shifts/data/shift_repository.dart`).	Set `create, update: if false`.
S4	One admin level, no audit trail.	`isAdmin()` checks a single claim. `getUserDetail` (`admin.go`) returns the entire private profile. `storage.rules` lets any admin read every document.	Roles plus an append-only audit log (Phase 2).
S5	Sensitive data has no safe home.	`GetProfile` returns the whole profile to its owner and `GetUserDetail` to any admin. `DBSCertificateNumber` already sits on the profile. NI number and bank details would land there next.	Separate server-only collections, field-level encryption, masked by default, every read audited (Phase 4 and 8).

ID	Finding	Evidence	Fix
S6	All functions share one broad service account.	`PRODUCTION_SETUP.md` section 7 grants the default compute account `datastore.user`, `firebaseauth.admin`, `storage.objectAdmin`, `pubsub.editor` and more.	Give payments its own service account with only what it needs (Phase 10).

3.3 Further security gaps

ID	Finding	Evidence	Fix
S7	No App Check and no rate limiting.	`CONFIG_AND_KEYS.md`.	Enable App Check in monitor mode first; enforce only after old app builds are retired.
S8	Uploaded documents are kept indefinitely and unscanned.	Storage rules check only size and content type. No retention rules exist.	Retention schedule and delete jobs (Phase 4).
S9	The design docs overstate protections.	`ARCHITECTURE.md` versus the rules file (S2).	Add a rules test for every server-owned field so a regression fails CI.

3.4 Reliability and operations

ID	Finding	Effect
R1	Every backend file exists twice (service root and `function/`), with no CI and manual `gcloud` deploys. **Confirmed identical today.**	One forgotten copy changes behaviour between local and production.
R2	No staging project. `KNOWN_ISSUES.md`: "any change... is tested directly against production."	Every release is a live experiment.

ID	Finding	Effect
R3	Pub/Sub events are published after the Firestore commit and failures are only logged (`cancelShift`, `decideApplicant`). **Confirmed.**	A lost event is a lost notification or chat session. Acceptable for chat, not for money or payroll.
R4	`MatchNewShift` publishes one message per practitioner profile for every new shift. **Confirmed.**	Cost and notification spam grow with users. It ignores location, vetting and suspension.
R5	No check for overlapping shifts. `acceptShift` reads only the target shift. **Confirmed.**	A practitioner can be booked twice at the same time. The owner explicitly wants double bookings prevented.
R6	No scheduler anywhere in the repo. **Confirmed.**	Trial, recheck and shift reminders and weekly invoicing cannot run.
R7	No email sending. Only in-app and push.	The owner asked for email notifications.
R8	Money is `float64` (`payRate`) with no pence, currency or rate snapshot.	Rounding errors and retroactive rate changes.
R9	`ListAllUsers`, `GetPlatformStats` and `ListPendingDocuments` scan whole collections.	Fine for hundreds of users; slow and costly beyond that.
R10	No monitoring or alerts, backups unverified, test admin account still present (the seed script defines one with a known password).	Outages and data loss would go unnoticed.
R11	Old app builds treat unknown enum values as defaults: unknown status becomes `open` (`shifts/domain/shift.dart`), unknown `dbsStatus` becomes `unverified`.	Adding a new shift status such as `completed` would show finished shifts as open in old apps. New states must be new fields.

3.5 Where the earlier Gemini plan was wrong or incomplete

- It listed booking transactions as missing. They already exist in `acceptShift`, `cancelShift`, `markNoShow` and `decideApplicant`. The real gap is overlapping shifts (R5).

- It treated `AcceptShift` as acceptance. It is the application step, and the nursery's approval is the booking.
- It listed fields as missing that exist: `DBSCertificateNumber`, `RegisteredCompanyName`, `OfstedRegNumber`, `Postcode`.
- It recommended Stripe Connect Express. K Flex pays practitioners through PAYE payroll, so Connect is not needed. `backend/services/payments/TODO.md` still says Connect and should be rewritten.
- It started the trial at profile creation. The owner's draft terms say the trial begins on an activation date, which should follow admin approval.
- It missed S1 to S3, R1, R2 and R11, and rated Security and Cancellations as "substantial".

4. Requirements coverage: owner's brief versus the code

Of the owner's 15 sections, none is complete, six are missing or only a stub, and nine are partial. The table uses the section numbers from the owner's *K Flex Changes Oct 2026* brief. "Phase" points to section 8.

#	Owner requirement	Status	What exists today	What is missing	Phase
1	Subscriptions: 14-day trial, Standard £199, Premium £299, Pay as you go £9.99, countdown, editable plans, renewals, cancel, upgrade, VAT	Missing	Nothing. No subscription data or code.	Everything, including admin-editable plans and discount codes.	9 (charging in 10)
2	Payments: Stripe, subscription and shift payments kept separate, bank transfer, invoices, receipts, refunds, reconciliation, weekly invoicing	Missing	`services/payments` stub returning 501. `shifts.paymentStatus` enum is never used. `transactions` name is reserved and locked.	Everything.	10 (manual invoice export in 9)

#	Owner requirement	Status	What exists today	What is missing	Phase
3	Practitioner payroll: editable rates, rolled-up holiday pay with an eligibility flag, payroll records, CSV export, payslips, earnings history	Missing	One `payRate` float on each shift.	Rate tables, holiday calculation, payroll periods and references, export, payslip storage, earnings screen.	8
4	DBS: dedicated section, Update Service link, admin checklist, never auto-verify, "DBS Checked by K Flex", recheck reminders	Partial	Certificate image upload, admin approve or reject, `DBSCertificateNumber`, `DBSExpiryDate`, a verified badge.	Structured fields, consent, checklist, link in app, badge wording and date, recheck reminders, statuses and notes. Remove the auto-verify (S1).	1 and 4
5	Practitioner registration: personal, employment, verification and payroll details, three statuses, request more documents, approval before accepting shifts	Partial	Staff wizard (basics, experience, documents). Document types: DBS, CV, ID, qualification, first aid, right to work. Optional training gate.	Date of birth, full address, NI number, bank details, references, employment history, preferred locations and hours, safeguarding certificate, the three statuses, request-more-documents, and the approval gate.	4
6	Nursery registration: full details, multiple branches, admin approve or reject	Partial	Wizard collects name, address, postcode, phone, email, registered company name, Ofsted registration number and rating, facilities.	Company number, billing details, preferred payment method, number of locations, staffing requirements, branches, approval and rejection.	5

#	Owner requirement	Status	What exists today	What is missing	Phase
7	Shift booking: location, required qualification, breaks, safeguarding information, hourly charge, matching notifications, expected gross earnings, no double bookings	Partial	Create and edit shifts (title, times, capacity, `payRate`, age group, room, children, duties, requirements). Apply, approve, reject, transactional capacity, chat.	Location and branch, required qualification, breaks, safeguarding and health and safety fields, nursery charge versus practitioner pay, expected earnings, overlap check, real matching.	6
8	Timesheets: start, finish, submit, nursery approve or correct, admin disputes, link to invoices and payroll, wages not withheld	Missing	`MarkNoShow` only. "Completed" is derived, not stored.	Everything.	7
9	Cancellations and no-shows: separate nursery and practitioner flows, editable policy, charges, replacements	Partial	`CancelShift` for both parties with no policy, reason or timestamp. Staff cancel notifies the nursery. `MarkNoShow`.	Policy configuration, charge calculation, reasons and times, admin notification, replacement flow, emergency and sickness handling.	6
10	Admin dashboard: metrics, rate and price editors, discounts, refunds, exports, roles, audit log	Partial	Four tabs: counts, document queue, user list and detail with suspend, badges, training editor.	Subscription, revenue and payroll metrics, editors for prices, rates and policies, discount codes, refunds and credits, notification sender, exports, role-based access, audit log, legal document upload.	2, then each phase

#	Owner requirement	Status	What exists today	What is missing	Phase
11	Terms and policies: six separate documents, versioned acceptance, re-acceptance, drafts not published	Missing	Static `terms.html`, `privacy.html` and `support.html` on Hosting. Links in `legal_links.dart`.	Six policies, versions with a draft, approved and published workflow, acceptance records, acceptance gate, re-acceptance, engagement agreement and Key Information Document.	3
12	Notifications: full nursery and practitioner lists, email and in-app	Partial	In-app and push for chat, applied, decided, booked, cancelled, rating, matched.	Email, a scheduler, and every other listed event.	2, then each phase
13	Security and data protection	Partial	Auth, owner-scoped rules (with gaps S2 and S3), Storage rules, account deletion, suspension that revokes tokens.	S1 to S9, encryption of sensitive fields, retention and deletion rules, backup verification.	1, 2, 4, 8, 11
14	Company information: editable fields, nothing published until given	Missing	Placeholders in static pages.	Editable settings with a publish flag per field.	2
15	Final testing, including failed payments, expired subscriptions, double bookings, rejected applications	Partial	Go trigger tests and 22 rules tests. No Flutter tests, no end-to-end suite.	End-to-end suite on staging and the 17 scenarios in section 9.	0, 11

What already exists and should be reused, not rebuilt

- Transactional apply, approve and cancel logic in `booking.go`, including auto-rejection when a shift fills.
- The notification pipeline: Pub/Sub topics, `notifications` documents, FCM push and the in-app notification centre.

- Chat, ratings, the training engine with server-side scoring, account deletion and suspension with token revocation.
- The admin panel scaffolding in `admin_review_screen.dart` and its repository layer.
- The Remote Config update gate (`app_update_service.dart`), which becomes the compatibility lever in section 6.
- The profile and public profile split, where nothing is public unless it is explicitly mirrored.
- The Workload Identity Federation set-up in the Android workflow, which CI deploys should reuse.

5. Decisions and defaults

Fourteen decisions shape the build. Each has a default that is safe to build on today, so work never blocks on the owner. The last column says what it costs to change later: **config** means an admin setting or flag, **small code** means a contained change, **structural** means it touches the data model or flow and is worth confirming before the named phase. I am not a lawyer or payroll adviser; rows marked "adviser" need professional input before the app goes live with real people.

#	Decision	Why it matters	Build now on this default	If the owner differs	Needed by
1	Is the iOS app live on the App Store, and how is Android distributed?	`KNOWN_ISSUES.md` says iOS is not live (TestFlight secrets and APNs key missing) and Android is a direct APK; the earlier plan says the App Store. Sets how many old app versions exist.	Assume users on 1.6.0+9 or older exist. Raise `min_supported_version_code` only after the new client is adopted.	Config	Phase 0
2	Who sets hourly rates? Today the nursery types `payRate`. The brief has the admin set practitioner pay.	Decides the shift data model and what practitioners see.	Admin sets practitioner pay (£17 or £16 gross). The nursery does not type a rate. Each shift stores both numbers at booking time. Keep the legacy `payRate` populated for old apps.	Structural	Phase 6

#	Decision	Why it matters	Build now on this default	If the owner differs	Needed by
3	What does a nursery pay per hour? The brief gives practitioner pay and fees but no nursery charge.	Needed to compute invoices and expected margin.	Configurable rate card per qualification, with an optional per-nursery override. Seed with placeholder charges that the owner replaces; invoices stay manual until then.	Config	Phase 6
4	Is K Flex VAT-registered, and are £199, £299, £9.99 and staffing charges inclusive or exclusive of VAT? (adviser)	Changes every displayed price and invoice.	Store amounts excluding VAT plus a VAT mode flag. Show VAT only when a VAT number has been entered and published.	Config	Phase 9
5	Do practitioners "accept" and get booked at once (the brief's wording), or apply and the nursery approves (what exists)?	Changes the booking flow and the vetting gate.	Keep apply then nursery approval. Relabel the button. Add optional instant booking per nursery later.	Structural	Phase 6
6	Employment set-up: who is eligible for rolled-up holiday pay, how Agency Workers Regulations equal-treatment rules apply, and what the Key Information Document and engagement agreement say. (adviser)	The system must not assume eligibility, and pay rates may change.	Per-worker eligibility flag set only by an admin with payroll permission, default off.	Config	Phase 4

#	Decision	Why it matters	Build now on this default	If the owner differs	Needed by
7	Payroll software. GOV.UK describes Basic PAYE Tools as software for businesses with fewer than 10 employees, which K Flex will likely exceed. GOV.UK does not say whether it supports pensions or file import. (adviser)	The export format and the "free payroll" cost assumption both depend on this.	Build a generic payroll CSV with one row per practitioner per pay period. Choose payroll software that can import it before launch. Confirm pension duties.	Small code	Phase 8
8	Payment provider and methods. Stripe is the owner's preference.	Card fees are high for £199 to £299 plans, and Bacs Direct Debit is usually cheaper.	Stripe Billing and Invoicing with card plus Bacs Direct Debit. Price GoCardless as an alternative before committing. Launch on bank transfer first.	Small code, isolated in `payments`	Phase 10
9	Where nurseries buy subscriptions. Apple has rules on in-app purchases.	Could force a different payment flow on iOS.	Take billing on the web through hosted checkout and keep the app free of purchase buttons. Verify the current App Store guidelines before building.	Structural	Phase 9
10	Admin surface: keep the admin panel inside the mobile app, or add a small web admin?	Payroll exports, finance tables and audit logs are awkward on a phone.	Add a minimal web admin on Firebase Hosting using the same claims.	Small code	Phase 2
11	What happens when a nursery does not approve a timesheet? The brief says wages must not be withheld.	Needs a payable-hours rule that does not depend on the nursery.	After 48 hours without a nursery response, the timesheet goes to the admin queue. Payroll uses admin-approved hours.	Config (the 48 hours is a setting)	Phase 7

#	Decision	Why it matters	Build now on this default	If the owner differs	Needed by
12	Who gets a free trial?	Prevents repeat signups for repeat trials.	Trial starts on admin approval of the nursery. One trial per company number or per name and postcode. Admin can override.	Config	Phase 9
13	Cancellation notice periods and fees.	The brief says charges apply only per the approved policy.	Ship with all fees at zero. Admin edits the policy and publishes a new version before fees apply.	Config	Phase 6
14	Retention periods for DBS data, right-to-work copies, payroll records and rejected applications. Also whether to record IP addresses with policy acceptances. (adviser)	Deletion jobs and the privacy policy depend on them.	Retention periods configurable; no automatic deletion until the owner approves them. Record timestamp and app version, not IP.	Config	Phase 4

Source for row 7: Basic PAYE Tools on GOV.UK, opened by the original author on Oct 23, 2026.

What cannot go live on a default

These are built and tested, but stay off or unpublished until the owner supplies the content: the final text of the six policies (drafts are never published), the company number, registered address and VAT number (hidden until entered and published), cancellation fees (zero), rolled-up holiday pay eligibility (off for everyone), and live Stripe keys. Nothing in this list blocks the app from working; each one just shows nothing, or the safe zero, until it is filled in.

6. Rules of engagement for every phase

Every change must keep today's app working for users who never update, and each rule below is binding in all phases. A model picking this up should read this section before touching code.

6.1 Compatibility: do not disrupt live users

1. **Additive only.** Add fields, collections and endpoints. Never rename, remove or retype an existing field or endpoint, and never change what an existing value means.
2. **Never add values to an enum that old apps parse.** Old apps map unknown `shifts.status` to `open` and unknown `dbsStatus` to `unverified` (R11). A new lifecycle state is always a new field or a separate document, for example a `timesheets` document, never `status: completed`.
3. **New gates fail politely.** A blocked action returns the standard error envelope with a stable `code` and a human-readable `message`, for example `VERIFICATION_REQUIRED`, `TERMS_ACCEPTANCE_REQUIRED`, `TRIAL_EXPIRED`. Old apps show the message instead of crashing.
4. **Every gate has three modes: `off`, `warn`, `enforce`.** The mode lives in `platformConfig/featureFlags`, is read by the server and cached for 60 seconds. Deploy the gate in `off`, move to `warn` (log who would have been blocked), then `enforce`. Ship the client screens before enforcing.
5. **Grandfather deliberately.** Migrations set explicit statuses for existing users. Where a gate would lock out people with live bookings, give them a configurable grace date.
6. **Use the version gate.** Raise Remote Config `min_supported_version_code` only after the new client is out and adopted. The forced-update screen already exists.
7. **Test rules against the real client.** Before deploying any rules change, run the current production build against the staging project. `auth_repository.dart` is the only direct Firestore write, and it writes `role`, `name`, `dbsStatus: unverified`, `rating`, `createdAt` and later `photoUrl`. Check git history for older builds that wrote other fields.
8. **Migrations are scripts, not requests.** Put them in `backend/migrations/`: idempotent, with `--dry-run`, batched, resumable and logged. Run on staging first.
9. **Defaults over waiting.** When an owner decision is open, build on the default in section 5, behind a flag that is off or in `warn`. A working app on a default beats a blocked one.

6.2 Backend conventions

- **Two copies of every file.** Until Phase 0 removes the duplication, change both the service root and `function/`, then run `diff -r` between them.
- **A new function needs four edits:** the handler's `init()` registration, `deploy.sh`, `run-local.sh` (a port) and the `ApiFunction` enum in `frontend/lib/core/config/env.dart`.
- **A new Pub/Sub topic must exist before its trigger deploys:** `gcloud pubsub topics create <topic>`.
- **Money is `int64` pence with currency `GBP`.** Never `float64`. Round half-up per line item. Compute holiday pay as gross minus basic so the parts always sum to the gross.
- **Time is UTC in storage.** Business-day logic uses `Europe/London`.
- **Invariants live in transactions.** Capacity, overlap checks, status changes and their outbox events are written in one Firestore transaction. Reads come before writes.
- **Idempotency.** Endpoints with money, payroll or notification side effects accept an `Idempotency-Key` header. Store the key and response in `idempotency/{key}`. Stripe webhooks use the event id the same way.
- **Outbox for events that must not be lost.** For money, payroll, legal and safeguarding events, write `outbox/{id}` in the same transaction as the change. A trigger publishes it and marks it sent. Consumers de-duplicate on event id. Chat and ratings can stay best-effort.
- **Dead-letter topics with retry backoff** on every Pub/Sub subscription, and an alert on dead-letter depth.
- **Validate everything server-side.** Reject unknown enum values, cap lengths, and never trust a client-supplied role, status or staff id.
- **Stop leaking internals.** Several handlers return `err.Error()` in 500 responses. Return a generic message and log the detail.
- **Structured logging** with `log/slog`, including a request id. Never log names, NI numbers, bank details, DBS numbers or document paths.
- **Config reads** are cached in memory for 60 seconds and carry a version field. If a read fails, use the last good value.

6.3 Security conventions

- **Rules deny by default.** Every server-owned field is protected by an allowlist (`affectedKeys().hasOnly([...])`), and every one has a rules test.

- **Sensitive data is server-only.** NI number, bank details, date of birth, DBS number and vetting evidence live in separate collections with `allow read, write: if false`. Encrypt sensitive fields with a Cloud KMS key (key ring in `europe-west2`), decrypt only inside functions that hold the right permission, and mask by default in the admin UI.
- **Admin access to sensitive files** goes through a function that checks the permission, writes an audit entry and returns a short-lived signed URL. Direct Storage reads by the admin claim stay only for non-sensitive prefixes.
- **Admin roles** use a new claim `adminRole`: `super_admin`, `verifier`, `finance`, `support`, `read_only`. Keep `admin: true` set for compatibility with the current app.
- **Audit log.** `auditLogs/{id}` is append-only and denied to all clients. Each entry records time, actor uid and role, action, target, the names of changed fields (not values, for sensitive data), request id and app version.
- **App Check in monitor mode first.** Enforce only after old builds are retired. Add rate limits on write endpoints with a small per-uid limiter.
- **Secrets live in Secret Manager**, never in the repo. `payments` runs as its own least-privilege service account.
- **CI scans:** `govulncheck` for Go and dependency alerts for Flutter and npm.
- **Owner tasks outside the code:** a data protection impact assessment (children's sector, DBS data) and ICO registration. Check both with an adviser.

6.4 Availability and performance

- **Cold starts:** measure p95 latency first. If needed, set minimum instances to 1 on a few hot endpoints (`ListOpenShifts`, `AcceptShift`, `GetProfile`, `SendChatMessage`, timesheet submit), not on every function.
- **Firestore:** no new full-collection scans. Use aggregation count queries for admin stats, cursor pagination for lists, and add each composite index to `firestore.indexes.json` and deploy it before the code that needs it. Do not update one hot document many times per second.
- **Scheduled jobs:** Cloud Scheduler calls an HTTP function and the function verifies the OIDC token. Every job is idempotent, resumable, logs a summary and alerts on failure.
- **Backups:** enable Firestore point-in-time recovery, add a daily export to a bucket with a 30-day lifecycle, turn on Storage object versioning for documents, and rehearse a restore once.

- **Monitoring:** alert policies for 5xx rate and p95 latency per function, dead-letter depth, scheduler failures and Firestore errors, plus an uptime check. Add Crashlytics to the app.
- **Degrade gracefully:** a failed notification never fails the main action, and the app shows cached data on timeouts.

6.5 Testing and release discipline

- **Every pull request runs:** `go vet` and `go test ./...` per service, `npm run rules:test`, `flutter analyze` and `flutter test`, and the copy-drift check.
- **Tests each phase must add:** table-driven money tests (golden values: £17.00 gives £15.17 basic and £1.83 holiday; £16.00 gives £14.28 and £1.72), emulator tests that fire parallel applications at one shift, a rules test for every frozen field and every new collection, and contract tests that replay the requests the current production client sends.
- **Release order:** additive rules and indexes, then backend, then client, then turn on the flag, then much later raise the minimum app version.
- **Rollback:** tag every deploy; roll back a function by shifting traffic to its previous Cloud Run revision; use the flag to disable a gate instantly. Because migrations are additive, old code keeps working.
- **Small pull requests** (a few hundred lines) behind flags. Each states what changed, which flag controls it, how to roll back and what test evidence exists.

6.6 Definition of done for a task

- Both source copies updated and identical.
- Tests added and passing locally and in CI.
- Rules and indexes updated and tested.
- Verified against the current production client build on staging.
- Behind a flag, with default off.
- Audit entries written for any admin, payment, payroll or verification change.
- No new personal data in logs.
- Rollback step written in the pull request.

7. Target data model (additive only)

Every item below is a new collection, a new field, or a new endpoint. Nothing renames or retypes an existing field (rule 6.1.1). Sensitive collections are server-only (`allow read, write: if false`).

Area	New collection or field	Notes
Config	`platformConfig/featureFlags`	One map of gate name to `off`, `warn` or `enforce`. Cached 60 s.
Config	`platformConfig/company`	Editable company fields, each with `value` and `published` (bool). Public pages show only published fields.
Admin	Claim `adminRole`	`super_admin`, `verifier`, `finance`, `support`, `read_only`. Keep `admin: true`.
Audit	`auditLogs/{id}`	Append-only, server-written. Changed field names only for sensitive data.
Events	`outbox/{id}`, `idempotency/{key}`	Outbox written in the same transaction as the change.
Legal	`policies/{type}/versions/{n}`	`status`: `draft`, `approved`, `published`. Only `published` is shown or required.
Legal	`policyAcceptances/{id}`	uid, policy type, version, timestamp, app version, status. No IP (decision 14 default).
Practitioner	`profiles/{uid}.vettingStatus` (new field)	`pending_verification`, `approved`, `rejected_further_info`. Never reuse `dbsStatus` for this.

Area	New collection or field	Notes
Practitioner	`vetting/{uid}` (server-only)	Checklist: identity, original certificate, suitability, workforce, barred list, Update Service, right to work, qualification, references, safeguarding, first aid. Each item has result, checker uid, date, note.
Practitioner	`vetting/{uid}/requests/{id}`	Requests for more documents.
Practitioner	`sensitive/{uid}` (server-only, KMS-encrypted fields)	NI number, bank details, date of birth, DBS number, address.
Practitioner	`profiles/{uid}.dbsCheckedByKFlex` and `dbsVerifiedAt`, `dbsRecheckDue`	Written only by the vetting function. Mirrored to `profilesPublic`.
Practitioner	`payrollProfiles/{uid}` (server-only)	`rolledUpHolidayEligible` (default false, finance role only), qualification tier.
Nursery	`nurseries/{id}` and `nurseries/{id}/branches/{bid}`	Company number, billing details, preferred payment method, branch address and geohash.
Nursery	`profiles/{uid}.nurseryApprovalStatus` (new field)	`pending`, `approved`, `rejected`.
Shift	`shifts/{id}.branchId`, `requiredQualification`, `breakMinutes`, `safeguardingNotes`, `nurseryArea`	All optional, so old shifts keep working.
Shift	`shiftFinancials/{shiftId}` (server-only)	Pence amounts snapshotted at booking: `practitionerGrossPence`, `basicPence`, `holidayPence`, `nurseryChargePence`, `rateCardVersion`.

Area	New collection or field	Notes
Rates	`rateCards/{id}`	Per qualification: practitioner gross, nursery charge, optional per-nursery override. Versioned.
Cancellation	`cancellationPolicies/{version}` and `shifts/{id}/cancellations/{id}`	Reason, actor, timestamp, computed charge (zero until a policy is published).
Timesheet	`timesheets/{id}`	Separate document, not a shift status. `submitted`, `approved`, `correction_requested`, `disputed`, `admin_approved`.
Payroll	`payrollPeriods/{id}`, `payrollLines/{id}`	Pence only. Reference, period, status, paid date.
Billing	`plans/{id}`, `subscriptions/{nurseryId}`, `invoices/{id}`, `discountCodes/{id}`	Amounts ex VAT plus VAT mode. Invoice numbers from a transactional counter.
Payments	`transactions/{id}`	Written by payments service only.

Gate model

A practitioner may apply only when all hold: `vettingStatus == approved`, not suspended, accepted current practitioner terms, no overlapping booked shift (checked in the same transaction), and required training complete if that flag is on. A nursery may post or approve only when `nurseryApprovalStatus == approved`, not suspended, accepted current nursery terms, and trial or subscription active (or pay-as-you-go). Every gate returns the standard error envelope with a stable code and runs in `off`, `warn` or `enforce` (rule 6.1.4).

8. Phased implementation plan

Work one phase at a time. Do not start a phase until the previous one's acceptance checks pass. Every phase follows sections 6.1 to 6.6. Each phase is a series of small pull requests behind flags. Open owner decisions never block a phase: build on the section 5 default, and change it when the owner answers.

Phase 0: Safety net (no behaviour change)

Goal. Make change safe before changing anything.

1. **Remove duplicated source trees (R1).** Choose one source of truth per service. Either build from `function/` using a deploy-time copy step, or make the service root a thin entrypoint importing the package. Until then add a CI job that fails on `diff -r` between root and `function/`.
2. **CI for the backend.** On every pull request: `go vet`, `go test ./...` per service, `govulncheck`, `npm run rules:test`, plus `flutter analyze` and `flutter test`.
3. **Create a staging Firebase project** with the same region. Copy the setup in `PRODUCTION_SETUP.md` into a script. Seed it with `make seed`.
4. **Scripted deploys.** One script per service, tagged releases, rollback by moving Cloud Run traffic to the previous revision. Deploy from CI to staging, from a manual approval to production, authenticating through Workload Identity Federation as the Android workflow already does (no service account keys).
5. **Feature flags.** Create `platformConfig/featureFlags` and a cached server reader with a last-good fallback.
6. **Operations.** Enable Firestore point-in-time recovery, a daily export with a 30-day lifecycle, Storage object versioning for document prefixes, alert policies (5xx, p95 latency, scheduler failures, Firestore errors) and an uptime check. Add Crashlytics. Rehearse one restore on staging. Rotate or delete the leftover test admin account.
7. **Contract tests.** Record the requests the current production client (1.6.0+9) sends and replay them in CI. Resolve decision 1 (iOS and Android distribution).
8. **Docs.** Refresh README, ARCHITECTURE.md and the payments TODO (PAYE payroll, not Connect).

Acceptance. CI is green and required on `main`. A deliberate drift between copies fails CI. A staging deploy and a rollback have both been performed. A restore rehearsal is written up.

Phase 1: Fixes and rules

Goal. Close S1 to S3 and fix B1 to B4. Smallest possible behavioural change for honest users.

1. **S2.** Replace the `profiles/{uid}` update rule with an allowlist of client-writable fields using `affectedKeys().hasOnly([...])`. On create, allow only the fields `auth_repository.dart` writes today (`role`, name, `dbsStatus: unverified`, `rating`, `createdAt`, and later `photoUrl`), and require `dbsStatus == 'unverified'`, `suspended` absent or false, and every trust field absent. Check git history for older builds that wrote other fields (rule 6.1.7).
2. **S3.** Set `create, update: if false` on `shifts`. Confirm in staging that the production client never writes shifts directly.
3. **S1.** Stop `reviewDocument` from setting `dbsStatus` or `rightToWorkVerified`. Existing verified users keep their value (explicit migration, rule 6.1.5). Document approval now only records the document status. Until Phase 4 ships, the admin sets verification through a temporary admin-only endpoint that writes an audit entry. Announce this to the owner, since it changes the admin routine.
4. **B1 to B4 and the quiz invalidation.** As described in section 3. Add the parity test for the two public-profile builders.
5. **Tests.** One rules test per frozen field (S9), including create-time checks.

Flags. None needed for rules. S1 change is reversible by redeploy.

Acceptance. A client write to `identityVerified`, `ofstedVerified`, `rightToWorkVerified`, `suspended`, `role` (after creation), or a shift fails in the emulator. The production client build still signs up, edits its profile and applies to a shift on staging. B1 to B4 have regression tests.

Phase 2: Platform foundations

Goal. The plumbing every later phase needs.

1. **Admin roles and audit log.** Add the `adminRole` claim, a `requireRole(ctx, permission)` helper, and `writeAudit()` called inside the same transaction as each admin change. Keep `admin: true`. Backfill existing admins as `super_admin`.
2. **Outbox and idempotency helpers.** Shared package under `backend/shared`. Add a dead-letter topic and retry backoff on every subscription, plus an alert on dead-letter depth.

3. **Cloud Scheduler.** One HTTP function per job, OIDC verified, idempotent, logs a summary, alerts on failure. Start with a no-op heartbeat job.
4. **Email.** Pick a provider (decision pending; keep the cost low). Add a `SendEmail` outbox consumer in `communication`, templates stored in code, an unsubscribe-safe footer for non-essential mail, and no personal data in logs (R7).
5. **Company information.** `platformConfig/company` with a per-field `published` flag. Legal pages read only published fields (owner's section 14).
6. **Minimal web admin** on Firebase Hosting (decision 10), reusing the same claims. Start read-only (stats, audit log viewer), then add each admin screen as its phase needs it.
7. **App Check in monitor mode** (S7). Add a per-uid rate limiter on write endpoints.
8. **Stats without full scans (R9).** Replace `GetPlatformStats` scans with aggregation count queries.

Acceptance. An admin action writes exactly one audit entry. A killed consumer leaves outbox items that are retried and de-duplicated. A scheduler job runs on staging and alerts when forced to fail. An email arrives from staging.

Phase 3: Legal documents and acceptance

Goal. Owner's section 11. Six policies, versioned, nothing published without approval.

1. Collections `policies` and `policyAcceptances`. Types: nursery terms, practitioner terms, privacy, cancellation and refund, safeguarding, cookie. Later add engagement agreement and Key Information Document.
2. Admin (role `super_admin`) uploads a draft, marks it `approved`, then `published`. Drafts are never served to the public or required for acceptance. The owner's drafts contain placeholders such as `[REGISTERED COMPANY NAME]`; block publishing while any bracketed placeholder remains.
3. Endpoints `GetRequiredPolicies` and `AcceptPolicies` (idempotent). Record user id, type, version, timestamp, app version and status.
4. Client: an acceptance screen in both registration wizards and a re-acceptance screen when a new version is published. Ship it before enforcing.
5. Hosting pages (`terms.html`, `privacy.html`, `support.html`) read published versions and company fields instead of static text.
6. Gate `TERMS_ACCEPTANCE_REQUIRED` in `off`, then `warn`, then `enforce` for new registrations first. For existing users give a grace date.

Acceptance. A new user cannot finish registration without accepting the current published versions. Publishing a new version triggers a notification and, with the gate on, a re-acceptance prompt. A draft is invisible to the public.

Owner input. Final legal text, reviewed by an adviser.

Phase 4: Practitioner vetting and sensitive data

Goal. Owner's sections 4, 5 and 13. No practitioner applies for shifts without admin approval, and DBS is never auto-verified.

1. **Sensitive store.** Create the Cloud KMS key ring in europe-west2. Add `sensitive/{uid}` with encrypted fields and a decrypt helper available only to the vetting and payroll functions. Migrate `DBSCertificateNumber` out of `profiles` (copy, switch reads, then clear), with `--dry-run`.
2. **Registration additions.** Date of birth, full address, employment history, references, preferred locations and hours, DBS structured fields (certificate number, issue date, type and workforce, Update Service status, consent), safeguarding and first aid certificates. Add the Update Service link opening `https://www.gov.uk/dbs-update-service` in the app. Payroll fields (NI number, bank details) go to `sensitive/`, never `profiles`.
3. **Vetting checklist and statuses.** Endpoints for verifiers: record each check, request more documents, approve, reject. Approval sets `vettingStatus: approved`, `dbsCheckedByKFlex`, `dbsVerifiedAt` and `dbsRecheckDue`, with an audit entry. This replaces the Phase 1 temporary endpoint. Qualification verification records whether the qualification suits the role and ratios.
4. **Sensitive file access.** A function checks the permission, writes an audit entry and returns a short-lived signed URL. Remove the blanket admin read from sensitive Storage prefixes.
5. **Recheck reminders** via a Scheduler job and the email consumer.
6. **Retention.** Configurable retention periods (decision 14) and a deletion job that stays disabled until the owner approves periods.
7. **Migration.** Existing practitioners are set explicitly: those already `dbsStatus: verified` and with a reviewed document become `approved` only after an admin confirms the checklist (list them in a CSV for the verifier); everyone else `pending_verification`. Give live bookings a grace date.
8. **Gate** `VERIFICATION_REQUIRED` in `acceptShift` and in `decideApplicant`, in `off`, `warn`, `enforce`.

Acceptance. Entering a DBS number or approving a document never changes `vettingStatus`. An unapproved practitioner gets `VERIFICATION_REQUIRED` from the HTTP function and cannot bypass it through Firestore. No NI or bank detail appears in any log, profile read, or admin list. Every sensitive read has an audit entry.

Owner input. Decisions 6 and 14. Start the DPIA and ICO registration now (6.3).

Phase 5: Nursery registration and approval

Goal. Owner's section 6.

1. Extend the nursery wizard with company number, billing details, preferred payment method, number of locations and staffing requirements.
2. `nurseries/{id}` with `branches/{bid}`. Existing nurseries get one branch created from their current address by an idempotent migration.
3. `nurseryApprovalStatus` with admin approve and reject endpoints, reasons and audit entries. Approval sets the activation date used by the trial in Phase 9.
4. Gate `NURSERY_NOT_APPROVED` on `CreateShift` and `decideApplicant` (`off`, `warn`, `enforce`). Grandfather existing nurseries as `approved`.
5. Notifications: registration confirmation, approval and rejection.

Acceptance. An unapproved nursery cannot post or approve. Branches can be added and selected on a shift. Existing nurseries keep working unchanged.

Phase 6: Shifts, rates, matching and cancellations

Goal. Owner's sections 7 and 9.

1. **Rate cards** (decisions 2 and 3). Admin edits per-qualification practitioner pay and nursery charge, with versions. At booking time, write `shiftFinancials` in pence from the active card; the nursery no longer types `payRate`, but keep the legacy field populated for old apps.
2. **Shift fields.** Branch, required qualification, breaks, safeguarding and health and safety notes. Practitioners see expected gross earnings (hours minus breaks, times their snapshotted rate).
3. **Overlap check (R5).** In the apply and approve transactions, query the practitioner's booked and pending shifts for time overlap and fail with `SHIFT_OVERLAP`. Add an emulator test that fires parallel applications at overlapping shifts.

4. **Matching (R4).** Replace per-profile fan-out with a filtered query: approved, not suspended, qualification, location radius via geohash, availability. Batch and cap messages. Add a feature flag and compare with the old matcher in `warn` mode.
5. **Relabel** the button to Apply (decision 5). Keep apply then nursery approval.
6. **Cancellations.** `cancellations` documents with reason, actor and UTC timestamp. Policy editor with all fees at zero until the owner publishes a version (decision 13). Nursery cancellation shows the policy and records the time. Practitioner cancellation notifies the nursery and admin and creates a replacement task. Emergency and sickness reasons are separate. The charge calculator is pure and unit tested, and reads only the published policy.
7. **Keep `CancelShift` working** for old apps; new behaviour is added through new endpoints and fields.

Acceptance. Two parallel applications to overlapping shifts yield exactly one success. Rates are snapshotted and unaffected by later rate changes. A published policy with zero fees produces zero charges.

Phase 7: Timesheets

Goal. Owner's section 8, without touching shift status (R11).

1. `timesheets/{id}` created when a booked shift ends. States: `submitted`, `approved`, `correction_requested`, `disputed`, `admin_approved`.
2. Practitioner records start and finish and submits. Nursery approves, corrects or reports an attendance issue. Admin resolves disputes.
3. **Payable hours rule** (decision 11): after 48 hours without a nursery response, the timesheet moves to the admin queue and payroll uses admin-approved hours. Wages are never held back because a nursery has not approved or paid.
4. Scheduler jobs for reminders and the 48-hour escalation. Idempotent.
5. Completed shifts and approved hours for the admin dashboard come from timesheets, not from the derived check.

Acceptance. An unresponsive nursery cannot block payment. Each state change writes an audit entry and an outbox notification.

Phase 8: Practitioner payroll

Goal. Owner's section 3. Exports for external PAYE software, not a replacement for it.

1. `payrollProfiles` with the eligibility flag (default off, finance role only). Rolled-up holiday pay is computed only when the flag is on.
2. Money in `int64` pence, round half up per line, holiday pay computed as gross minus basic. Golden tests: £17.00 gives £15.17 and £1.83, £16.00 gives £14.28 and £1.72.
3. `payrollPeriods` and `payrollLines` from admin-approved timesheet hours, with reference, status and paid date. CSV export, one row per practitioner per period (decision 7). Excel is optional.
4. Practitioner earnings screen with completed shifts and gross history. Payslips are uploaded by an admin to a private path and linked to the practitioner, with a notification.
5. Everything audited. Exports are role-gated and logged.

Acceptance. Non-eligible workers show no holiday pay. Re-running a period is idempotent. The sum of parts always equals gross.

Owner input. Decisions 6 and 7 (adviser).

Phase 9: Subscriptions and invoicing without Stripe

Goal. Owner's sections 1 and 2 short of card payments. This lets the owner launch.

1. `plans` editable from admin: price in pence, billing interval, booking allowance, premium benefits, active flag. Seed Standard £199, Premium £299, Pay as you go £9.99 per booking. Amounts stored ex VAT plus a VAT mode flag (decision 4); show VAT only when a VAT number has been entered and published.
2. **Trial** starts on admin approval of the nursery and lasts 14 days (decisions 12 and 5). The dashboard shows start, expiry and days left. Scheduler jobs notify before expiry. No automatic charge. Staffing hours during a trial are still chargeable.
3. `subscriptions/{nurseryId}` with status, plan, renewal date. An admin can assign or change a plan; the gate `TRIAL_EXPIRED` applies after expiry unless the nursery has an active plan or uses pay-as-you-go.
4. **Invoices.** Transactional invoice numbers, weekly invoice generation for approved timesheets, PDF export, paid or unpaid status marked by an admin for bank transfer, reconciliation notes, outstanding payment notifications.
5. **Discount codes**, refunds and credits recorded as ledger entries only (no money movement yet).
6. Billing is web-only (decision 9); verify current App Store guidelines before any in-app purchase UI. The app shows status only.

Acceptance. A nursery moves through trial, expiry, an admin-assigned plan and a bank-transfer invoice end to end on staging. Subscription charges and staffing charges stay on separate invoice lines.

Phase 10: Stripe

Goal. The last feature phase.

1. Decision 8: price Stripe Billing with card and Bacs Direct Debit against GoCardless before committing.
2. Give `payments` its own least-privilege service account and keep keys in Secret Manager (S6). Add the Stripe SDK only to this service.
3. Hosted checkout for subscriptions, with the amount and VAT shown before authorisation. No automatic charge unless the nursery clearly authorised a paid plan.
4. Webhooks verified by signature, de-duplicated on event id, writing `transactions` and updating `subscriptions` and `invoices` through the outbox.
5. Failed payment notifications and retry state. Refunds issued from the admin only, with audit entries. No card data touches our database.
6. Run Stripe in test mode on staging for at least one full billing cycle simulation before live keys.

Acceptance. Replayed, duplicated and out-of-order webhooks leave consistent state. A failed payment produces a notification and a visible status, and never silently cancels a booking.

Phase 11: Hardening and end-to-end testing

Enforce App Check only after old builds are retired (raise `min_supported_version_code` first). Run the 17 scenarios in section 9 on staging, plus load tests on the hot endpoints, a second restore rehearsal, a dependency and secret scan, and a review of every flag so each is deliberately set. Measure p95 latency and set minimum instances on only the hot endpoints that need it (6.4).

9. Test scenarios

Run these end to end on staging with the current production client and the new client. The first 17 are the owner's list; A to D are the extra cases the owner named.

#	Scenario	First testable in	What to assert
1	Practitioner registration	Phase 4	All required fields captured; sensitive ones land in `sensitive/`; status is `pending_verification`.
2	Nursery registration	Phase 5	Branches created; status `pending`.
3	Document upload	Phase 4	Type and size checks; owner-only access; retention tag.
4	Admin verification	Phase 4	Checklist required; DBS number alone never verifies; audit entry.
5	Trial activation	Phase 9	Starts on nursery approval; 14 days; countdown correct.
6	Subscription selection	Phase 9	Amount and VAT shown before authorising; no auto-charge.
7	Payment processing	Phase 10	Webhook idempotency; no card data stored.
8	Shift posting	Phase 5 and 6	Unapproved nursery blocked; branch, qualification and breaks stored.
9	Practitioner booking	Phase 4 and 6	Unvetted blocked with `VERIFICATION_REQUIRED`; capacity and overlap hold under parallel load.
10	Shift completion	Phase 7	Timesheet created; shift status unchanged.
11	Timesheet approval	Phase 7	Correction, dispute and 48-hour escalation paths.
12	Invoice generation	Phase 9	Pence totals; subscription and staffing lines separate; unique numbers.
13	Payroll export	Phase 8	Golden values; eligibility flag respected; CSV columns match the owner's list.
14	Cancellation process	Phase 6	Time recorded; zero fees until a policy is published; both parties and admin notified.

#	Scenario	First testable in	What to assert
15	Notifications	Phases 2 to 10	In-app and email, each event in the owner's list, no duplicates on retry.
16	Account security	Phases 1, 2, 4	Rules tests for every frozen field; role checks; no sensitive data in logs.
17	Terms acceptance	Phase 3	Record has user, time, version and status; re-acceptance after a new version.
A	Failed payments	Phase 10	Notification, retry, no silent loss of access to confirmed bookings.
B	Expired subscriptions	Phase 9	`TRIAL_EXPIRED` shown with a readable message; old app does not crash.
C	Double bookings	Phase 6	Parallel overlapping applications give one success.
D	Rejected practitioner applications	Phase 4	Rejected status, reason and request-more-documents flow; cannot apply.

Extra automated checks for every phase: contract replay of the production client, rules tests for each new collection, parallel-write emulator tests for any new invariant, and a migration dry-run diff.

10. Short answers for the owner

1. **Completed:** marketplace core (apply, approve, capacity, chat, ratings), basic profiles and documents, admin queue, training modules, suspension.
2. **Missing:** subscriptions, payments, payroll, structured DBS vetting, the three practitioner statuses with an approval gate, nursery approval and branches, timesheets, cancellation policy, legal acceptance, email, audit log and roles, company settings.
3. **Before launch:** Phases 0 to 8 and the manual-billing parts of Phase 9. Phases 1 and 4 are the safeguarding and data-protection essentials.

4. **Payment provider:** Stripe is a reasonable default, but card fees are heavy on £199 and £299 plans. Compare Stripe with Bacs Direct Debit and GoCardless before committing. Launch can use bank transfer first.
5. **Admin dashboard:** the current four-tab panel cannot cover this. Add a small web admin with roles and an audit log (Phase 2) and extend it phase by phase.
6. **Extra costs to expect:** an email service, a staging project, Cloud KMS, Scheduler and monitoring (small at this scale), payroll software that can import CSV, Stripe or GoCardless fees, and professional help for legal text, payroll and holiday-pay eligibility, DPIA and ICO registration. I have not priced these here; check current provider pricing before quoting the owner.

Open items

The build proceeds on the section 5 defaults; the owner's answers then change settings or small code. Decisions 2, 5 and 9 are structural, so get them confirmed before Phases 6 and 9. Decisions 1, 6, 7, 8 and 14 affect cost and compliance most. The first piece of work is Phase 0.

Appendix A: Verification log and extra findings

- **Re-pin the commit.** The review was done at commit `3a79a61`; `main` of Bridge-Flex was still at that commit when I cloned the repo on Oct 23, 2026 (latest commit dated Sep 24, 2026). Before each phase, run `git diff 3a79a61..HEAD --stat` to see whether anything changed since, and re-verify the findings that phase depends on.
- **The README is stale.** The root README still lists 22 endpoints, says there is no profile wizard and describes the app as a first-accept-wins marketplace. The review (35 functions, wizard, apply-then-approve) reflects the code. Treat code and rules files as the source of truth, and refresh README, ARCHITECTURE.md and backend/services/payments/TODO.md in Phase 0.
- **Local dev uses plain HTTP and emulators.** `make dev`, `make seed` and `make dev-with-triggers` exist and the devharness runs the real trigger handlers in-process. Reuse them for the new tests in Phase 0.
- **Seeded test accounts use the password `password123`.** Make sure none of these exist in production (R10).

- **Scope of this document's verification.** I cloned the repo at 3a79a61 on Oct 23, 2026 and checked these against the code, and all held: S1 (documents.go sets dbsStatus verified on approval), S2 (profile rules freeze only dbsStatus, rating and training ids; create forbids only the training field), S3 (clients can create and update shifts), B2 (the public-profile builder omits training ids), B3 (the course counter counts unpublished modules), B4 (listMyShifts never queries pendingStaffIds and swallows errors), R1 (the duplicated trees are currently byte-identical), R4 (matching reads every staff profile), R5 (no overlap check), no Cloud Scheduler anywhere, no backend or analyze CI (workflows only build and screenshot), 22 rules tests (8 + 7 + 7), no frontend/test folder, and version 1.6.0+9. ARCHITECTURE.md lines 176 to 182 say suspended, identityVerified and ofstedVerified are frozen, which the rules do not do. New: backend/seed/main.go defines an admin account (admin@example.com, password password123); confirm it was never seeded into production. Not re-checked: B1 line numbers in detail, and anything about the live project (backups, IAM, Remote Config values, deployed rules).
- **More from the full read (new, not in sections 2 to 6).** (a) KNOWN_ISSUES.md still says there is no profile wizard, but the staff and nursery onboarding screens exist; update it. (b) S2 is sharper than the review says: admin.go already has a server-side write path for identityVerified and ofstedVerified, and the profile rules let a client write the same fields without it. (c) Storage rules let any signed-in user read profile-photos and nursery-photos, and any admin read every dbs-documents and personal-documents file; there is no split between sensitive and non-sensitive prefixes. (d) The Android workflow uses Workload Identity Federation and its comments say the project org policy forbids service account keys, so CI deploys must use the same mechanism, not JSON keys. (e) Firebase client API keys appear in google-services.json, GoogleService-Info.plist and firebase_options.dart. These are public by design, but confirm each key is restricted to the app and APIs it needs in the GCP console. (f) KNOWN_ISSUES.md says point-in-time recovery is unverified and iOS is not live (the TestFlight secrets are missing), which matches decision 1. (g) There is no backend/migrations folder yet, and no Idempotency-Key or outbox code; both are new in Phase 2.